

The University of Azad Jammu and Kashmir, Muzaffarabad

Name: Muneeba Arshad

Roll NO: 2024-SE-06

Course Title: Digital Image Processing

Instructor Name: Engr. Fahad Nisar

Lab: 04

Semester: 5th

Session: 2024-2028 (SE)

Department of Software Engineering

LAB REPORT — TASK 03

Image Enhancement and Intensity Transformations

Objective and Setup

This lab calculates, implements and compares grayscale intensity transformations, histogram enhancement and thresholding. Every operation was first worked out by hand in a dry run, then implemented in Python (NumPy and OpenCV) and checked against the hand calculation with assertions. All images are displayed with fixed grayscale limits 0–255 so that visual comparisons are fair.

Numeric-safety rules followed

- The image is loaded as 8-bit grayscale and kept unchanged (made read-only); every result is a new array.
- Arithmetic uses wider types (int16 or float64), results are clipped to 0–255 before conversion to uint8, and fractional values are truncated.

Input image

Property	Value
File	input_image.png (8-bit grayscale)
Shape	(400, 600) — 400 rows × 600 columns (240,000 pixels)
dtype	uint8
Minimum / Maximum	0 / 255



Figure 1: *Original grayscale image (espresso cup on a wooden table) — unequal lighting makes it a good test for local methods.*

1. Brightness Adjustment

Transformation: s = r + β with β = +40 (brighter) and β = −40 (darker). The calculation is done in int16, clipped to 0–255, then converted to uint8. A second implementation with nested loops for β = +40 was compared with the NumPy result.

Image	Mean	Std. dev.	Clipped pixels
Original	103.7	58.1	0.00 %
β = +40	142.8	56.2	4.43 % at 255
β = −40	67.3	53.0	18.53 % at 0



Figure 2: *Original, brighter (β = +40) and darker (β = −40) images.*

The nested-loop result is identical to the NumPy result (np.array_equal → True). Doing the same addition directly in uint8 would make 10,466 pixels wrap around (e.g. 250 + 40 → 34), turning bright areas dark:

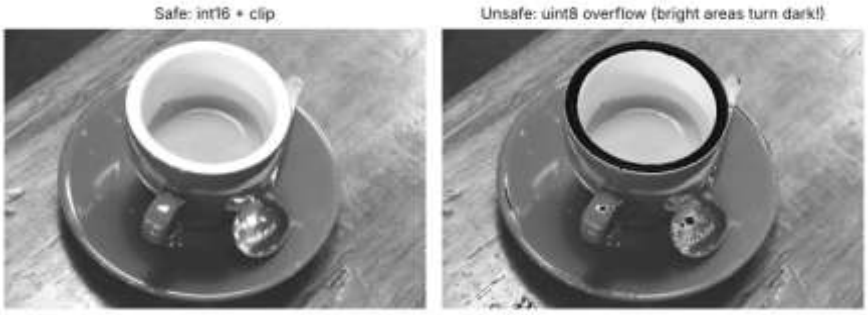


Figure 3: *Safe arithmetic (int16 + clip) versus unsafe uint8 overflow.*

Dry run 1 — pixels [30, 80, 128, 180, 240]

r	r + 40 (raw)	Clipped	uint8	r − 40 (raw)	Clipped	uint8
30	70	70	70	−10	**0**	0
80	120	120	120	40	40	40
128	168	168	168	88	88	88
180	220	220	220	140	140	140
240	280	**255**	255	200	200	200

> **Lost detail.** With +40, 240 → 280 is clipped to 255 and every original value ≥ 215 ends up as 255, so differences between bright pixels disappear. With −40, 30 → −10 is clipped to 0 and every value ≤ 40 becomes 0, so shadow detail is lost. Clipped information cannot be recovered.

Python verification: the uint8 results [70, 120, 168, 220, 255] and [0, 40, 88, 140, 200] match the hand calculation.

2. Linear Contrast from Zero

Transformation: $s = \alpha \cdot r + \beta$ with $\alpha = 1.5$ and $\beta = 0$, computed in float64, clipped and truncated.

The mean rises from 103.7 to 148.7 and the standard deviation from 58.1 to 75.8. **29,254 pixels (12.19 %)** saturate at 255 because every pixel with $r > 170$ gives $1.5 \cdot r > 255$.



Figure 4: *Original, s = 1.5·r, and the clipping mask (white = pixels clipped to 255).*

Dry run 2 — pixels [30, 80, 128, 180, 240]

r	1.5 · r	Clipped	Truncated	Saturated?
30	45.0	45	45	No
80	120.0	120	120	No
128	192.0	192	192	No
180	270.0	**255**	255	**Yes**
240	360.0	**255**	255	**Yes**

> **Observation.** Multiplying from zero makes the whole image brighter **and** raises contrast, but pixels 180 and 240 both become 255 and can no longer be told apart (highlight clipping).

Python verification: final values [45, 120, 192, 255, 255] and saturation flags match.

3. Contrast Centered at 128

Transformation: $s = (r - 128)\cdot\alpha + 128$ with $\alpha = 1.5$ and $\alpha = 0.5$, compared with linear contrast from zero $(1.5\cdot r)$.

Image	Mean	Std. dev.	% pixels at 0 or 255
Original	103.7	58.1	0.00
1.5·r (from zero)	148.7	75.8	12.48
Centered, α = 1.5	96.0	75.1	23.89
Centered, α = 0.5	115.6	29.1	0.00

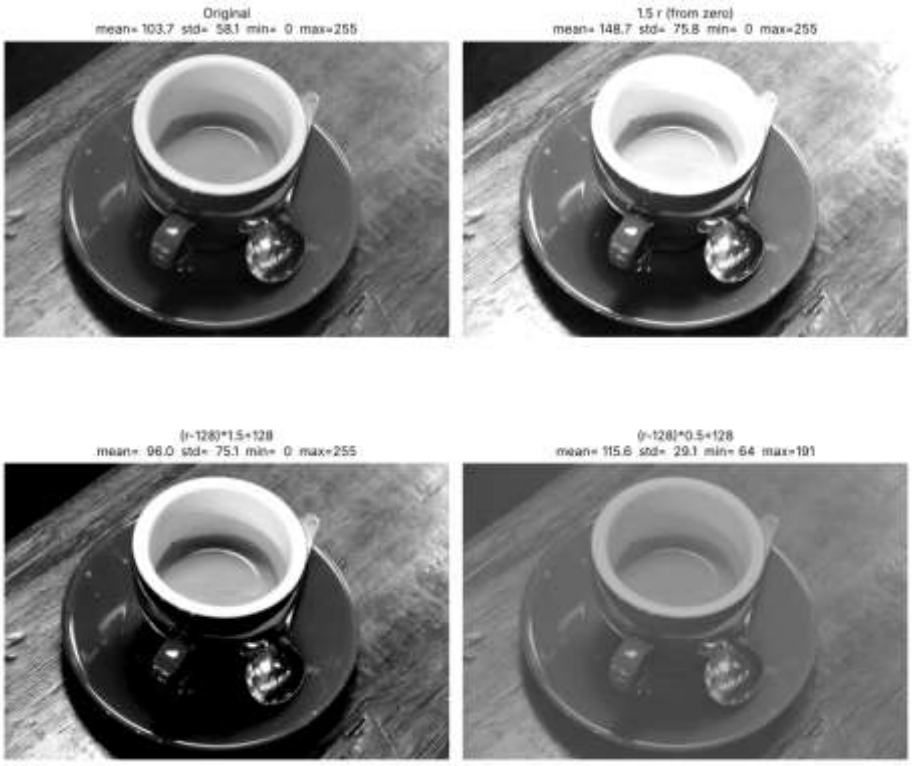


Figure 5: *Original, 1.5·r, centred α = 1.5 and centred α = 0.5.*

Dry run 3 — pixels [30, 80, 128, 180, 240]

α = 1.5

r	r − 128	× 1.5	+ 128	Clipped	Final
30	−98	−147	−19	**0**	0
80	−48	−72	56	56	56
128	0	0	128	128	128
180	52	78	206	206	206
240	112	168	296	**255**	255

α = 0.5

r	r − 128	× 0.5	+ 128	Clipped	Final
30	−98	−49	79	79	79
80	−48	−24	104	104	104
128	0	0	128	128	128
180	52	26	154	154	154
240	112	56	184	184	184

> **Result at r = 128.** (128 − 128)·α + 128 = 128 for every α, so 128 is the fixed pivot: contrast is stretched or compressed around mid-gray without moving it.
>
> **Observation.** Centred α = 1.5 increases contrast (std 75.1) while the mean stays close to the original (96.0), whereas 1.5·r also pushed the mean up to 148.7. α = 0.5 compresses all values toward 128 (std 29.1), giving a flat, washed-out image.

Python verification: [0, 56, 128, 206, 255] for α = 1.5 and [79, 104, 128, 154, 184] for α = 0.5 match.

4. Image Negative

Transformation: s = 255 − r. The mean changes from 103.7 to 151.3, and applying the negative twice returns exactly the original image (verified: True).



Figure 6: Original image and its negative.

> **When is it useful?** In medical imaging (e.g. X-rays or mammograms), fine bright details on a dark background are easier to inspect after inversion, because the eye separates small intensity differences better in dark regions than in bright ones.

Dry run 4 — pixels [30, 80, 128, 180, 240]

r	255 − r	Negative again: 255 − (255 − r)	Equals original?
30	225	30	Yes
80	175	80	Yes
128	127	128	Yes
180	75	180	Yes
240	15	240	Yes

Python verification: [225, 175, 127, 75, 15] and the double negative equals the original.

5. Gamma Correction

Transformation: s = 255·(r/255)^γ with γ = 0.5 and γ = 2.0 (full precision, final output truncated). **Convention:** γ is used directly as the exponent — γ < 1 brightens (expands dark tones) and γ > 1 darkens (compresses dark tones).

Image	Mean	Std. dev.	Mean in dark region (r < 64, 24.5 % of pixels)
Original	103.7	58.1	28.20
γ = 0.5	153.9	50.7	80.62
γ = 2.0	54.9	53.3	3.70



Figure 7: *Original, γ = 0.5 (brighter) and γ = 2.0 (darker).*

Applying both gamma values to [0, 255] returns [0, 255]: black and white stay fixed, so gamma correction never clips.

Dry run 5 — pixels [0, 64, 128, 192, 255]

γ = 0.5

r	r / 255	(r / 255)^0.5	× 255	Truncated
0	0.000000	0.000000	0.0000	0
64	0.250980	0.500979	127.7498	127
128	0.501961	0.708492	180.6654	180
192	0.752941	0.867722	221.2691	221
255	1.000000	1.000000	255.0000	255

γ = 2.0

r	r / 255	(r / 255)^2	× 255	Truncated
0	0.000000	0.000000	0.0000	0
64	0.250980	0.062991	16.0627	16
128	0.501961	0.251965	64.2510	64
192	0.752941	0.566920	144.5647	144
255	1.000000	1.000000	255.0000	255

> **Difference from a rounded slide example.** If the normalised value is rounded early (64/255 ≈ 0.25, 128/255 ≈ 0.50, 192/255 ≈ 0.75), the results become 127, 180, **220** for γ = 0.5 and **15, 63, 143** for γ = 2.0. Keeping full precision (64/255 = 0.25098, …) gives 127, 180, 221 and 16, 64, 144 — about one gray level different, caused purely by rounding before the power is applied.

Python verification: full-precision outputs [0, 127, 180, 221, 255] and [0, 16, 64, 144, 255] match; the rounded-normalization values were also reproduced.

6. Grayscale Histogram

A 256-bin histogram over the range 0–256 was computed with cv2.calcHist; the bin counts sum to 240,000, the total number of pixels.

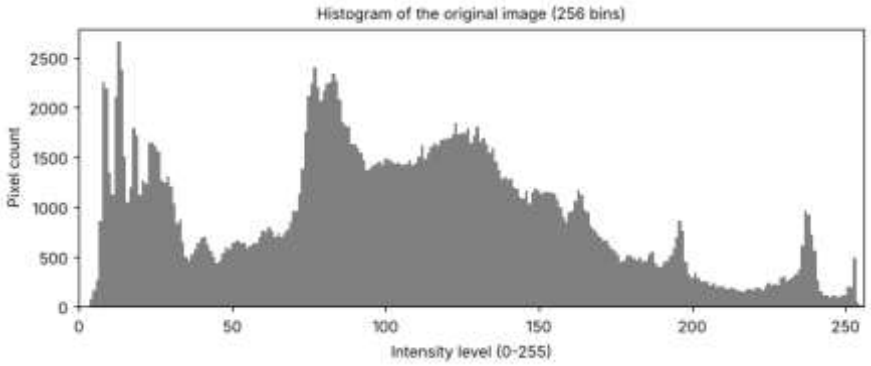


Figure 8: 256-bin histogram of the original image.

> **Interpretation.** The histogram has a mean of 103.7 and a standard deviation of 58.1, with a large group of dark pixels near 10–30 and a broad body around 70–160, so the image is overall mid-to-dark. About 24.5 % of the pixels are darker than 64 and only 7.6 % are brighter than 192, so although the full 0–255 range is used, most pixels sit in the dark-to-mid range.

Dry run 6 — A = [[0, 50, 50, 100], [100, 150, 200, 255]]

Intensity	Frequency	Probability (freq / 8)
0	1	0.125
50	2	0.250
100	2	0.250
150	1	0.125
200	1	0.125
255	1	0.125
Total	**8**	**1.000**

> **Why positions are not shown.** A histogram only counts how many pixels have each intensity. Any rearrangement of the same pixels (shuffling, flipping, rotating) produces exactly the same histogram, so all spatial information is discarded.

Python verification: frequencies [1, 2, 2, 1, 1, 1], total 8, probabilities sum to 1; a randomly shuffled copy of A gave an identical histogram.

7. Global Histogram Equalization

`cv2.equalizeHist` was applied to the grayscale image.

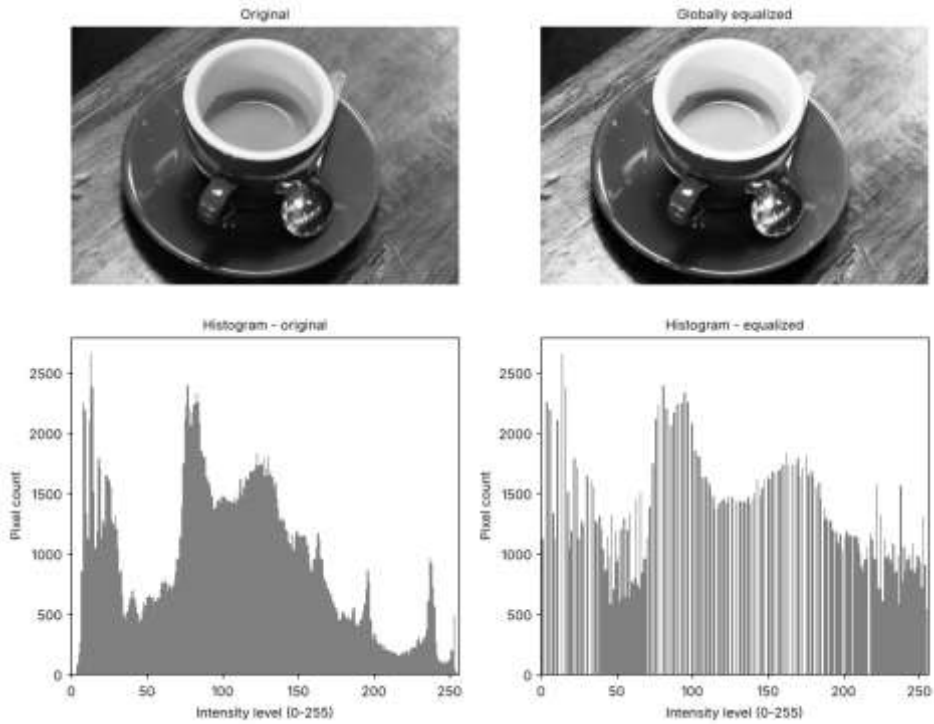


Figure 9: *Original and equalized images with their histograms.*

Measure	Original	Equalized
Mean	103.7	128.2
Standard deviation	58.1	73.5
Std. dev. in the smoothest 16 × 16 block	0.64	1.71
Mean |Laplacian| (fine detail + noise energy)	19.51	27.47

> **Discussion.** Equalization spreads the used gray levels over 0–255, so contrast and visible detail increase, particularly where levels were crowded together (the histogram becomes flatter, with gaps because the mapping is discrete). The same stretch also magnifies small

> variations in smooth areas: the smooth-block deviation rose from 0.64 to 1.71 and the Laplacian energy from 19.51 to 27.47, so **noise becomes stronger**.

Dry run 7 — pixels [50, 50, 100, 150] (N = 4)

Level	Frequency	Cumulative count	CDF
50	2	2	0.50
100	1	3	0.75
150	1	4	1.00

Level	Method 1: 255 × CDF	Method 2: 255 × (CDF − CDFmin) / (1 − CDFmin)
50	127.5 → 128	(0.5 − 0.5) / 0.5 = 0 → 0
100	191.25 → 191	(0.75 − 0.5) / 0.5 = 0.5 → 127.5 → 128
150	255	(1 − 0.5) / 0.5 = 1 → 255
Output list	**[128, 128, 191, 255]**	**[0, 0, 128, 255]**

> **Comparison with OpenCV.** CDFmin = 0.50 (lowest nonzero CDF); rounding is to the nearest integer with .5 rounded up. OpenCV returns **[0, 0, 128, 255]**, identical to Method 2. Method 1 never maps the darkest level to 0, so part of the output range is wasted; subtracting CDFmin fixes this.

Python verification: both output lists were reproduced and Method 2 equals `cv2.equalizeHist`.

8. CLAHE

cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)) was applied. The tuple (8, 8) is the **number of tiles** (8 rows × 8 columns), not the number of pixels per tile; for this 400 × 600 image each tile is about 50 × 75 pixels.

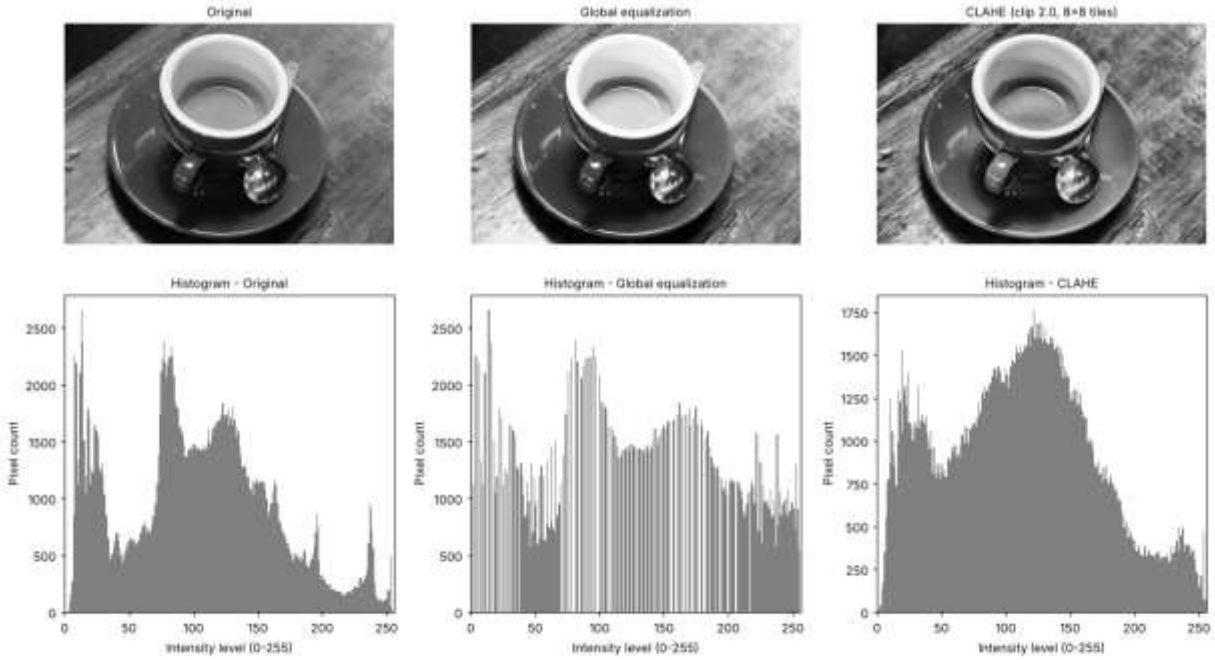


Figure 10: *Original, global equalization and CLAHE, each with its histogram.*

Mean / standard deviation per image quadrant (to examine uneven lighting):

Image	Top-left	Top-right	Bottom-left	Bottom-right
Original	110.8 / 57.1	144.8 / 39.8	79.4 / 63.6	79.6 / 41.4
Global HE	135.6 / 69.3	185.0 / 44.5	96.2 / 81.5	96.1 / 53.2
CLAHE	112.2 / 61.7	138.3 / 45.8	91.7 / 63.9	103.2 / 53.5

> **Comparison.** Global equalization applies one curve to the whole image, so the already-bright top-right quadrant is pushed much brighter (mean 144.8 → 185.0, risking washed-out highlights). CLAHE equalizes each tile separately: that quadrant stays near its original brightness (138.3) while its local contrast still rises (std 39.8 → 45.8), and dark regions are lifted. The clip limit caps how steep each tile's mapping can be, limiting noise amplification compared with unclipped adaptive equalization; however, the smooth-block deviation (0.64 →

> 1.57) and Laplacian energy (19.51 → 35.95) show CLAHE still increases fine-detail/noise energy — a trade-off against local detail.

Dry run 8 — histogram [6, 2, 0, 0], absolute cap = 4

Step	Bin 0	Bin 1	Bin 2	Bin 3	Total
Original counts	6	2	0	0	8
Clip at 4	**4**	2	0	0	6 (excess 2)
Add excess / 4 = 0.5 to each bin	4.5	2.5	0.5	0.5	8
Cumulative count	4.5	7.0	7.5	8.0	
New CDF	**0.5625**	**0.8750**	**0.9375**	**1.0000**	
Mapping × 255	143.4	223.1	239.1	255	

> **Effect of clipping.** Without clipping the CDF would be [0.75, 1, 1, 1] (× 255 → 191, 255, 255, 255), a very steep jump at the dominant bin. Clipping flattens that slope and limits contrast and noise amplification.
>
> **Why interpolation is needed.** Every tile has its own mapping function. If each pixel used only its own tile's mapping, neighbouring tiles would map the same gray level to different values and visible block edges would appear, so OpenCV bilinearly interpolates between the mappings of neighbouring tiles.

This simplified model uses an absolute cap and fractional redistribution; OpenCV's `clipLimit` is a relative factor (scaled by tile size and bin count) with integer redistribution, so its results differ slightly.

Python verification: redistributed counts [4.5, 2.5, 0.5, 0.5] and CDF [0.5625, 0.875, 0.9375, 1.0] match.

9. Fixed Binary Thresholding

cv2.threshold(img, 127, 255, cv2.THRESH_BINARY) gives 255 where r > 127 and 0 otherwise. The unique output values are **{0, 255}**, with **80,304 white pixels (33.5 %)**.

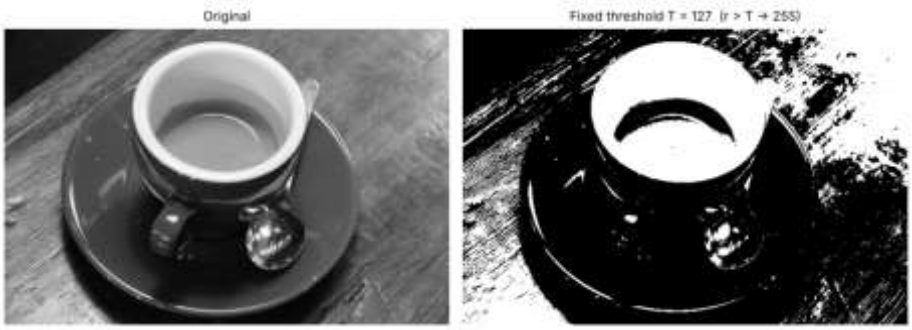


Figure 11: *Original image and fixed-threshold result (T = 127).*

Dry run 9 — pixels [0, 80, 127, 128, 180, 255] (rule: 255 if r > T else 0)

r	T = 127	T = 180
0	0	0
80	0	0
127	0 (127 > 127 is false)	0
128	**255**	0
180	**255**	0 (180 > 180 is false)
255	**255**	**255**
White pixels	**3**	**1**

> **Comparison.** Raising T from 127 to 180 reduces the white set from 3 pixels to 1; a pixel equal to T always goes to the background.

Python verification: results match `cv2.threshold` for both thresholds.

10. Otsu Thresholding

cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) returned **T = 105**. It gives 48.2 % white pixels, versus 33.5 % for the fixed T = 127. The two masks disagree on **35,419 pixels (14.76 %)** — the pixels with values between 105 and 127.

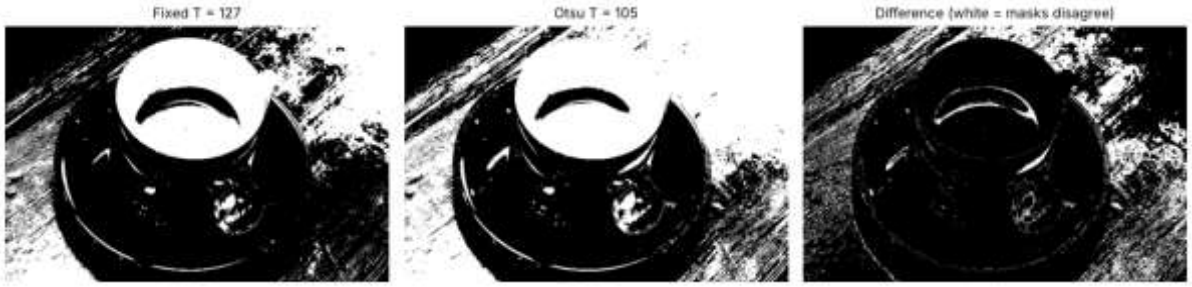


Figure 12: Fixed T = 127, Otsu T = 105 and the pixels where the two masks disagree.

> **Comparison.** Otsu chooses the threshold by maximising the between-class variance of this image's histogram, whereas T = 127 is arbitrary; here Otsu places the threshold lower and keeps more of the mid-gray structure white.

Dry run 10 — pixels [0, 1, 4, 5] (no rescaling), N = 4

Classes: class 0 = {r ≤ T}, class 1 = {r > T}; criterion $w0 \cdot w1 \cdot (\mu0 - \mu1)^2$.

T	Class 0	Class 1	w0	w1	μ0	μ1	$w0 \cdot w1 \cdot (\mu0 - \mu1)^2$
0	{0}	{1, 4, 5}	0.25	0.75	0	3.3333	2.0833
1	{0, 1}	{4, 5}	0.50	0.50	0.5	4.5	**4.0000**
2	{0, 1}	{4, 5}	0.50	0.50	0.5	4.5	4.0000
3	{0, 1}	{4, 5}	0.50	0.50	0.5	4.5	4.0000
4	{0, 1, 4}	{5}	0.75	0.25	1.6667	5	2.0833

> **Result.** The maximum value 4.0 is tied for T = 1, 2 and 3, so the smallest, **T = 1**, is chosen. The predicted mask (r > 1) is **[0, 0, 255, 255]**, and OpenCV returns threshold 1.0 with the same mask.

Python verification: the best T, the predicted mask and the OpenCV output agree.

11. Adaptive Gaussian Thresholding

cv2.adaptiveThreshold was applied with Gaussian weighting, binary output, blockSize = 11 and C = 2: each pixel is compared with its own threshold T = (Gaussian-weighted 11 × 11 mean) − 2.



Figure 13: Original, fixed (T = 127), Otsu (T = 105) and adaptive Gaussian thresholds.

Method	Threshold	White pixels
Fixed	127 (one value, global)	33.5 %
Otsu	105 (one value, global)	48.2 %
Adaptive Gaussian	local mean − 2 (per pixel)	64.5 %

The adaptive result agrees with Otsu on 56.1 % of the pixels and with the fixed threshold on 54.0 %.

> **When local thresholds help.** Fixed and Otsu thresholds use one value for the entire image, so under uneven lighting a dim but important region can fall completely below the threshold. The adaptive method follows local brightness and brings out edges and fine structure (wood grain, cup rim) — useful for shadows, uneven illumination and document scans. Because T lies slightly below the local mean, pixels in smooth areas tend to turn white, which explains the higher white percentage and speckle in flat regions.

Dry run 11 — 3 × 3 neighborhood

Neighborhood [[100, 100, 100], [100, 120, 140], [140, 140, 140]] with Gaussian weights [[1, 2, 1], [2, 4, 2], [1, 2, 1]] / 16:

Row	Calculation	Sum
1	1·100 + 2·100 + 1·100	400
2	2·100 + 4·120 + 2·140	960
3	1·140 + 2·140 + 1·140	560
Total	400 + 960 + 560	**1920**

Quantity	Value
Weighted mean	1920 / 16 = 120
Threshold T = mean − C	120 − 2 = 118
Centre pixel r = 120	**120 > 118 → 255 (white)**
A pixel equal to T (r = 118)	**118 > 118 is false → 0 (black)**

Python verification: weighted mean 120, T = 118 and both classifications match; OpenCV's centre pixel for the same patch (blockSize 3) is also 255. The 3 × 3 patch is used only for the dry run — the image task uses 11 × 11.

Summary of Results

#	Operation	Parameters	Key result
1	Brightness	β = ±40	Mean 103.7 → 142.8 / 67.3; 4.43 % clipped at 255, 18.53 % at 0
2	Linear contrast	α = 1.5, β = 0	Mean 148.7; 12.19 % of pixels saturate
3	Centered contrast	α = 1.5 / 0.5	Std 75.1 / 29.1; mid-gray 128 unchanged
4	Negative	255 − r	Mean 151.3; applying twice restores the original
5	Gamma	γ = 0.5 / 2.0	Means 153.9 / 54.9; 0 and 255 fixed
6	Histogram	256 bins	Mean 103.7, std 58.1; 24.5 % of pixels below 64
7	Global equalization	equalizeHist	Std 58.1 → 73.5; smooth-block std 0.64 → 1.71 (noise up)
8	CLAHE	clip 2.0, 8 × 8 tiles	Better local contrast without over-brightening the bright quadrant
9	Fixed threshold	T = 127	33.5 % white
10	Otsu	automatic	T = 105; 48.2 % white; differs from T = 127 on 14.76 % of pixels
11	Adaptive Gaussian	11 × 11, C = 2	64.5 % white; follows local lighting

Key Concepts

Clipping and overflow. Overflow is wrap-around: in uint8, 250 + 40 = 290 becomes 34 (290 mod 256), turning a bright pixel dark. Clipping (saturation) is the safe alternative: calculate in a wider type, force the result into 0–255 with np.clip, then convert to uint8 (fractions are truncated). Clipping loses detail because many values collapse to 0 or 255, but it never creates false values.

Brightness versus contrast. Brightness ($s = r + \beta$) shifts all intensities up or down; the spread is unchanged except where clipping occurs. Contrast ($s = \alpha \cdot r$ or $(r - 128) \cdot \alpha + 128$) scales the spread: $\alpha > 1$ increases it and $\alpha < 1$ reduces it. Scaling from zero also changes brightness, whereas scaling about 128 keeps mid-gray fixed.

Gamma convention. $s = 255 \cdot (r/255)^{\gamma}$ with γ used directly as the exponent: $\gamma < 1$ brightens by expanding dark tones, $\gamma > 1$ darkens by compressing them. 0 and 255 are fixed points, so no clipping occurs. (Display-gamma formulas often use $1/\gamma$, so the convention must always be stated.)

Global versus local methods. Global methods (equalization, fixed or Otsu thresholds) use one mapping or threshold for the whole image — fast and effective when lighting is uniform and the histogram is well separated. Local methods (CLAHE, adaptive thresholding) derive

mappings or thresholds from neighborhoods, which handles uneven illumination, shadows and local detail better, at the cost of more computation, parameter tuning (tile size, clip limit, block size, C) and possible noise amplification.

Conclusion

- All eleven transformations were calculated by hand, predicted, and verified in Python; every comparison with OpenCV matched.
- Safe arithmetic (wider types, clip, then convert) prevents overflow artefacts such as the 10,466 wrapped pixels shown in Section 1.
- Gamma and centered contrast changed tonality in a controlled way; linear contrast from zero and large brightness shifts caused measurable clipping.
- For this unevenly lit image, CLAHE and adaptive thresholding handled local lighting better than their global counterparts, while global equalization increased noise in smooth areas.